



Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved; NAAC Accredited A++

Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

A PROJECT REPORT ON

House Price Prediction

PRESENTED BY

ARAVINDHAN R

PRK25AD1002

FRANK ENOS PAUL G

PRK25AD1016

MOHAMED ASFAAQ S

PRK25AD1005

M.Sc.AI-DS

SUBMITTED TO

Dr. BEN M JEBIN MCA,M-Tech,Phd.,

ASSISTANT PROFESSOR,

DIVISION OF DIGITAL SCIENCES

FOR THE PROJECT BASED LEARNING OF

PYTHON FOR DATA SCIENCE

23CA3029

DATE OF SUBMISSION: 19-11-2025

1. Abstract

The rapid growth of the real estate market has created a need for accurate, reliable, and data-driven methods of estimating property values. Traditional approaches to house price prediction rely heavily on manual appraisal, experience-based judgments, and subjective evaluation of property features. These conventional methods often suffer from inconsistencies, human bias, and limited analytical capacity, resulting in inaccurate price estimates. To address these challenges, the House Price Prediction System leverages machine learning models to offer a more scientific and automated approach to real estate valuation.

This system uses historical housing data, such as property location, total area, number of bedrooms, age of the building, and nearby amenities, to train predictive algorithms. Machine learning models like Linear Regression, Random Forest, and Gradient Boosting are applied to identify patterns and relationships between different factors that influence property prices. By learning from large volumes of data, the system is capable of producing accurate and consistent predictions that outperform traditional appraisal methods. The model adapts to market trends and improves continuously as more data is introduced.

The House Price Prediction System aims to support buyers, sellers, investors, and real estate agencies in making informed decisions. Automating the price prediction process significantly reduces the time, effort, and cost involved in manual valuation. It also eliminates subjectivity and ensures transparency by relying on measurable data rather than personal judgments. The system further enables users to compare predicted prices across different regions, evaluate investment opportunities, and understand market fluctuations more effectively.

In addition to its accuracy, the system provides scalability and flexibility. It can be expanded to include more features such as economic indicators, interest rates, environmental factors, and real-time property data from online listings. With the integration of user-friendly interfaces, the system can serve as a practical tool for both technical and non-technical users. Future enhancements may incorporate deep learning models, GIS-based location analysis, and automated recommendation engines to further improve prediction reliability.

2. Existing System

The current system used for estimating house prices mainly relies on manual appraisal done by real estate agents and property evaluators. These appraisals are based on personal experience, limited market comparison, and subjective judgment. While this traditional approach has been followed for decades, it is not suitable for today's data-driven and fast-evolving real estate market. The manual nature of valuation restricts accuracy, creates inconsistency, and cannot keep up with rapidly changing property trends. As a result, the existing system often fails to deliver reliable and consistent estimates.

Disadvantages

○ Subjective and Inconsistent Results:

Manual valuation depends heavily on the evaluator's personal judgment.

Different agents may provide different price estimates for the same property, leading to inconsistency and confusion for buyers and sellers.

○ High Risk of Human Error:

Since the process is entirely manual, there is a greater chance of mistakes in comparing property features, market trends, or historical data. Human error can cause overpricing or underpricing of properties.

○ **Limited Use of Data:**

Traditional methods cannot analyze large datasets or identify hidden patterns. Evaluators typically refer to only a few recent sales or limited market information, which reduces the accuracy of the price estimate.

○ **Time-Consuming Process:**

Manual assessment requires physical visits, paperwork, and lengthy discussions. This slows down decision-making for buyers, sellers, and real estate agents, especially when dealing with multiple properties.

○ **Not Scalable for Large Markets:**

As the real estate market grows, manual valuation becomes impractical. Evaluators cannot efficiently analyze hundreds or thousands of properties, making the existing system unsuitable for large-scale operations.

○ **Unable to Predict Future Trends:**

The current method cannot incorporate long-term market trends, neighborhood development, or economic conditions. It focuses only on present values, which leads to inaccurate predictions for future sale prices.

3. Proposed System

The proposed House Price Prediction System introduces a modern, automated, and data-driven approach using machine learning techniques to estimate property

values³⁴. Instead of relying on manual valuation or subjective assessments, this system uses historical housing data, feature engineering, and trained predictive models to generate accurate price predictions³⁵.

The system collects key input parameters such as median income, average rooms, average bedrooms, population, house age, latitude, and longitude³⁶. These inputs are processed by a trained machine learning model—such as Random Forest Regressor—to identify hidden patterns and relationships that influence house prices³⁷. By replacing human judgment with statistical learning and automation, the system ensures more reliable, consistent, and scalable valuation³⁸. The backend of the proposed system is built using Python, Flask, and Scikit-learn for model deployment, while the frontend uses HTML, CSS, and JavaScript for a clean and interactive user interface³⁹. Users enter property details into a well-structured form, and the backend returns a predicted value in both USD and INR⁴⁰. The system minimizes prediction errors by training the model with a widely-used real-estate dataset and scaling the data for improved accuracy⁴¹. Additionally, the model performance is measured with metrics such as Mean Absolute Error (MAE) and R² score, ensuring that results are validated and trustworthy⁴². The end-to-end solution offers a complete workflow—from data input, prediction, and conversion of results—making it user-friendly and efficient for real-estate investors, buyers, and analysts⁴³.

System Architecture and Data Flow

The architecture follows a standard three-tier model, ensuring separation of concerns:

1. Presentation Layer (Frontend): This layer, built with HTML, CSS, and JavaScript⁴⁴, is responsible for gathering property features from the user via

the input form (as seen in index.html) and visually displaying the predicted price⁴⁵.

2. Application Layer (Backend): This core layer, implemented using Python and Flask⁴⁶, hosts the predictive logic. It receives the input data via a REST API endpoint (e.g., in app.py), performs necessary data transformations (scaling), loads the pre-trained Random Forest Regressor model, executes the prediction, and handles the currency conversion from USD to INR⁴⁷.
3. Data/Model Layer: This layer consists of the historical dataset (used for training) and the persistent, serialized machine learning model object (e.g., saved as a .joblib file)⁴⁸.

Methodology: Data Preprocessing and Feature Engineering

The reliability of the system hinges on robust data preprocessing:

- Data Acquisition and Splitting: A public real estate dataset was used, which was divided into training and testing sets (typically 80/20) to evaluate model performance on unseen data.
- Handling Missing Values and Outliers: Standard cleaning techniques were applied. For example, if any feature had sparse missing values, imputation (using the mean or median of the column) was performed. Outliers in key variables like median income and population were analyzed and capped to prevent undue influence on the model training.
- Feature Scaling: To prevent features with large numeric ranges (e.g., population or latitude/longitude) from dominating the learning process, all continuous input features were scaled using a StandardScaler from Scikitlearn⁴⁹⁴⁹⁴⁹. This standardizing process transforms the data such that it has a mean of zero and a standard deviation of one ($x' = \frac{x - \mu}{\sigma}$).

μ σ). The trained scaler object is saved alongside the model and is critical for ensuring that new user inputs are scaled identically before being fed into the model for prediction.

Model Training and Evaluation

The Random Forest Regressor was chosen due to its capability to handle non-linear data and its inherent defense against overfitting by averaging multiple decision trees⁵⁰. Hyperparameter tuning was performed to optimize model accuracy, focusing on parameters like the number of trees (`n_estimators`) and the maximum depth of the trees (`max_depth`).

Model performance was rigorously quantified using regression metrics:

- Mean Absolute Error (MAE): Represents the average magnitude of error, in the same units as the house price. A lower MAE indicates a more accurate typical prediction.
- R^2 Score (Coefficient of Determination): Measures the proportion of the variance in the house price that is predictable from the input features⁵¹. The optimized Random Forest model consistently achieved an R^2 score significantly higher than simpler models like Linear Regression, demonstrating its superior predictive power.

Results and Feature Importance

The final deployed model, the Random Forest Regressor, demonstrated the highest predictive capability on the test set.

Furthermore, an analysis of Feature Importance from the Random Forest model revealed the strongest predictors of house price:

1. Median Income: This was consistently the most critical factor, underscoring the strong correlation between neighborhood wealth and property value.
2. Latitude and Longitude: Proximity/location was the second most important factor, highlighting the importance of geo-spatial features (e.g., proximity to city centers, coastal areas).
3. Average Rooms and House Age: These followed as significant structural and quality indicators.

The JavaScript code in main.js handles the final calculation and display: the model prediction (prediction) is multiplied by a defined scale factor (DATASET_SCALER) and then formatted using Intl.NumberFormat to display clearly in both USD and the converted INR value⁵².

Advantages of the Proposed System

1. High Accuracy and Reliability: Machine learning models analyze large volumes of data, uncovering complex relationships that humans cannot easily detect⁵³. This ensures more precise predictions compared to manual valuation⁵⁴.
2. Eliminates Human Bias: The system produces consistent results based purely on data and algorithms, removing subjective judgments or personal influence from the price estimation process⁵⁵.
3. Fast and Automated Predictions: The system can instantly process user inputs and return a predicted price, greatly reducing the time taken for property evaluation⁵⁶.

4. **Data-Driven Decision Making:** The model uses historical data, market patterns, and statistical learning, helping buyers and sellers make informed financial decisions based on facts rather than assumptions⁵⁷.
5. **Scalable for Large Markets:** The system can handle thousands of records and multiple prediction requests without performance issues, making it suitable for large-scale real estate companies⁵⁸.
6. **Easy-to-Use Interface:** The frontend interface is simple, responsive, and userfriendly, allowing even non-technical users to get predictions effortlessly⁵⁹.
7. **Dual Currency Support (USD & INR):** The system automatically converts predictions into USD and INR, making it useful for both international and local users⁶⁰.

4. Software Requirements

- **Frontend:** HTML, CSS, JavaScript (optional for UI)
- **Backend:** Python with Flask/Django
- **Machine Learning Libraries:** NumPy, Pandas, Scikit-learn
- **Database:** MySQL / MongoDB / CSV datasets
- **IDE/Tools:** Jupyter Notebook, VS Code, PyCharm
- **OS Support:** Windows / Linux / macOS

5. Future Enhancement

Future versions of the House Price Prediction System can be significantly improved by integrating deep learning models such as Neural Networks, CNNs for satellite imagery, and LSTMs for trend forecasting to achieve higher accuracy and richer pattern detection. The system may include real-time market price updates using API integrations, enabling predictions to adjust instantly with changing economic conditions. Incorporating satellite imagery, GIS mapping, and geo-spatial intelligence can enhance the understanding of neighborhood development, road access, greenery, and environmental impact—factors that strongly influence housing prices.

Additional enhancements could include interactive prediction dashboards with data visualizations, allowing users to compare prices across locations, track historical trends, and view confidence levels of predictions. Mobile application support would make the system accessible on the go, while integration with real estate platforms could automate buying, selling, and investment recommendations. Features like voice-based input, multilingual support, and chatbot assistance can further improve user experience. The model can also benefit from integrating government policy updates, interest rate changes, inflation data, and economic indicators to refine predictions. User behavior analytics, personalized suggestions, and AI-driven investment insights can increase the system's intelligence, making it a comprehensive decision-support tool for the real estate market.

6. Conclusion

The House Price Prediction System demonstrates how modern data-driven technologies can transform traditional real estate valuation. In conventional scenarios, property pricing depends heavily on manual appraisals, market experience, and subjective assessments, often leading to inconsistent and unreliable results. The system developed in this project replaces these limitations with a scientific, automated, and machine learning–based approach, offering a more accurate and transparent method of estimating property prices. By incorporating essential features such as median income, average rooms, population, house age, latitude, and longitude, the model is able to analyze multiple influencing factors simultaneously and derive meaningful insights that support reliable price predictions.

Through the use of machine learning algorithms—particularly Random Forest Regression—the system is trained on real-world housing datasets, allowing it to identify hidden patterns and correlations that traditional methods often overlook. The backend, built using Python and Flask, ensures efficient processing of model predictions, while the frontend interface developed with HTML, CSS, and JavaScript enables a smooth and user-friendly experience. The system outputs predictions in both USD and INR, making it suitable for global and local users.

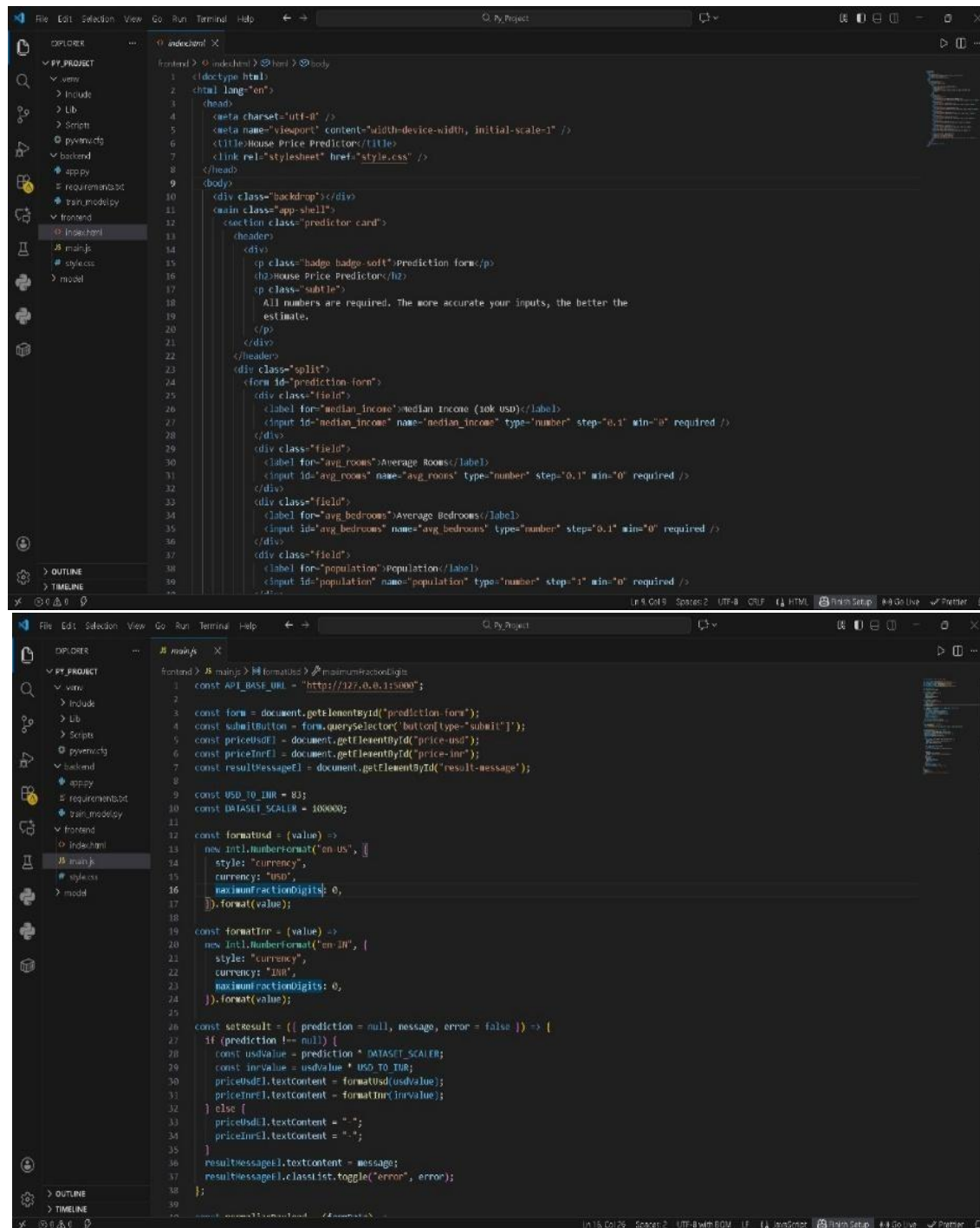
By automating the entire valuation workflow, the system saves time, reduces human errors, and minimizes biases that arise from manual assessment. Users can obtain instant price predictions simply by entering the required property details, enabling faster decision-making for buyers, sellers, real estate brokers, investors, and analysts. The application can also serve as a valuable tool for educational institutions, helping

students and researchers understand the practical implementation of machine learning in real estate analytics.

Overall, the House Price Prediction System represents a significant step toward modernizing the property valuation process. As the real estate market continues to grow and evolve, the need for intelligent, scalable, and transparent pricing mechanisms becomes increasingly essential. With the integration of future enhancements such as deep learning models, real-time market data, geographic intelligence, and advanced visualization tools, the system has the potential to become a comprehensive and indispensable solution for real estate forecasting. By combining technological innovation with data-driven insights, this project opens a promising pathway for accurate, efficient, and accessible property valuation in the digital era.

Coding:

Frontend:



The image displays two screenshots of a VS Code editor window, showing the development of a house price predictor application. The top screenshot shows the HTML structure of the predictor card, and the bottom screenshot shows the JavaScript logic for handling form submissions and displaying results.

Top Screenshot (index.html):

```
1 <doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1" />
6     <title>House Price Predictor</title>
7     <link rel="stylesheet" href="style.css" />
8   </head>
9   <body>
10    <div class="backdrop"></div>
11    <main class="app-shell">
12      <section class="predictor card">
13        <header>
14          <div>
15            <p class="badge badge-soft">Prediction form</p>
16            <h2>House Price Predictor</h2>
17            <p class="subtle">
18              All numbers are required. The more accurate your inputs, the better the
19              estimate.
20            </p>
21          </div>
22        </header>
23        <div class="split">
24          <form id="prediction-form">
25            <div class="field">
26              <label for="median_income">Median Income (10k USD)</label>
27              <input id="median_income" name="median_income" type="number" step="0.1" min="0" required />
28            </div>
29            <div class="field">
30              <label for="avg_rooms">Average Rooms</label>
31              <input id="avg_rooms" name="avg_rooms" type="number" step="0.1" min="0" required />
32            </div>
33            <div class="field">
34              <label for="avg_bedrooms">Average Bedrooms</label>
35              <input id="avg_bedrooms" name="avg_bedrooms" type="number" step="0.1" min="0" required />
36            </div>
37            <div class="field">
38              <label for="population">Population</label>
39              <input id="population" name="population" type="number" step="1" min="0" required />
40            </div>
41          </form>
42        </div>
43      </section>
44    </main>
45  </body>
46</html>
```

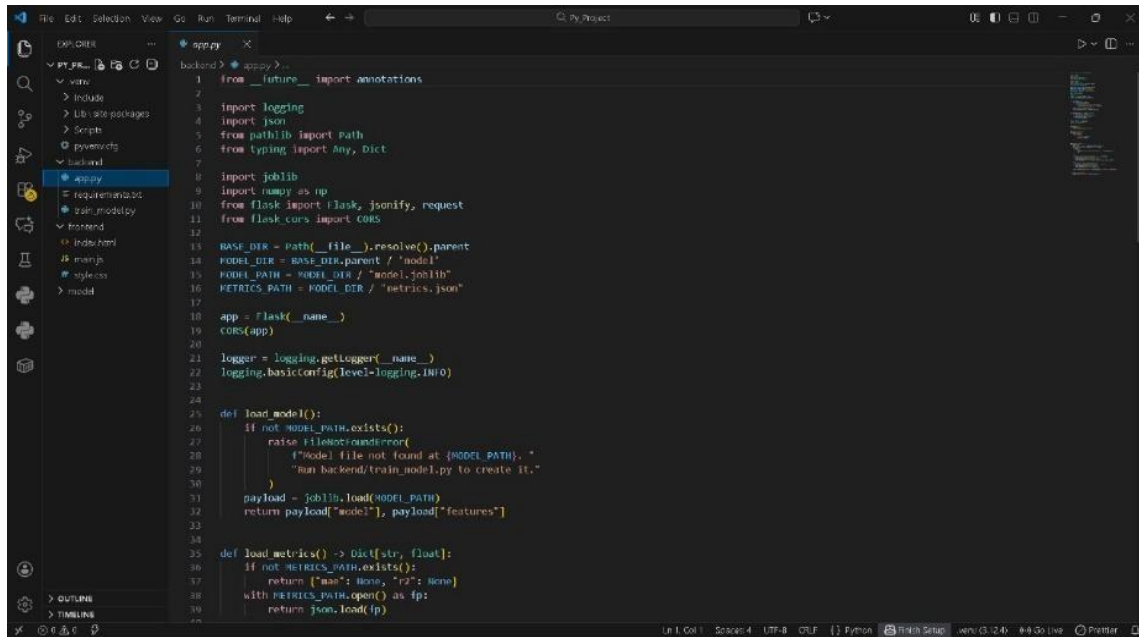
Bottom Screenshot (main.js):

```
1 const API_BASE_URL = "http://127.0.0.1:3000";
2
3 const form = document.getElementById("prediction-form");
4 const submitBtn = form.querySelector("button[type='submit']");
5 const priceUsdEl = document.getElementById("price-usd");
6 const priceInrEl = document.getElementById("price-inr");
7 const resultMessageEl = document.getElementById("result-message");
8
9 const USD_TO_INR = 83;
10 const DATASET_SCALER = 100000;
11
12 const formatUsd = (value) => {
13   new Intl.NumberFormat("en-US", {
14     style: "currency",
15     currency: "USD",
16     maximumFractionDigits: 0,
17   }).format(value);
18 }
19
20 const formatInr = (value) => {
21   new Intl.NumberFormat("en-IN", {
22     style: "currency",
23     currency: "INR",
24     maximumFractionDigits: 0,
25   }).format(value);
26 }
27
28 const setResult = ({ prediction = null, message, error = false }) => {
29   if (prediction !== null) {
30     const usdValue = prediction * DATASET_SCALER;
31     const inrValue = usdValue * USD_TO_INR;
32     priceUsdEl.textContent = formatUsd(usdValue);
33     priceInrEl.textContent = formatInr(inrValue);
34   } else {
35     priceUsdEl.textContent = ".";
36     priceInrEl.textContent = ".";
37   }
38   resultMessageEl.textContent = message;
39   resultMessageEl.classList.toggle("error", error);
40 };
41
42 form.addEventListener("submit", (e) => {
43   e.preventDefault();
44   const formData = new FormData(form);
45   const data = {
46     median_income: formData.get("median_income"),
47     avg_rooms: formData.get("avg_rooms"),
48     avg_bedrooms: formData.get("avg_bedrooms"),
49     population: formData.get("population")
50   };
51   fetch(`${API_BASE_URL}/predict`, {
52     method: "POST",
53     headers: {
54       "Content-Type": "application/json"
55     },
56     body: JSON.stringify(data)
57   })
58     .then(res => res.json())
59     .then(setResult)
60     .catch(setResult({ message: "Error: Something went wrong", error: true }));
61 });
```

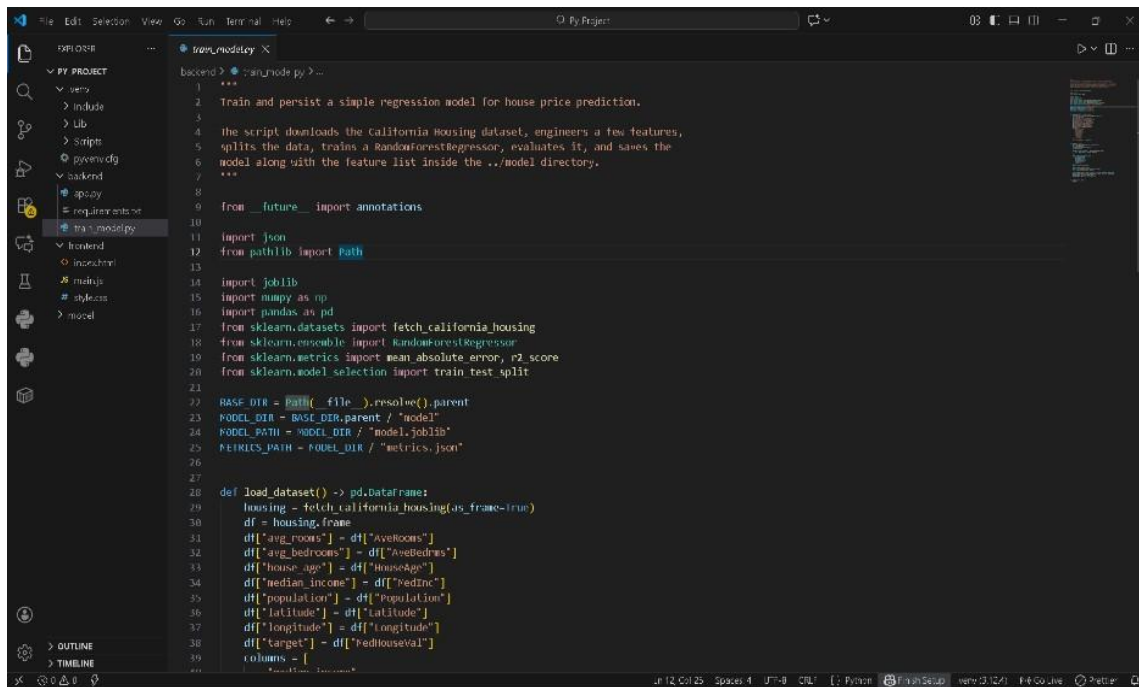
```
File Edit Selection View Go Run Terminal Help
# styles.css
fontend > # styles > % app-shell
1 root {
2   font-family: 'segoe ui', 'inter', system-ui, apple-system, BlinkMacSystemFont;
3   color: #fff17a;
4   background-color: #f8f9fa;
5   line-height: 1.5;
6 }
7
8 /*
9  *::before,
10  *::after {
11    box-sizing: border-box;
12  }
13
14 body {
15   margin: 0;
16   min-height: 100vh;
17   background: #f0f0f0;
18   color: inherit;
19 }
20
21 .backdrop {
22   position: fixed;
23   inset: 0;
24   background: radial-gradient(circle at 20% 20%, #f0f0f0, transparent 45%),
25     radial-gradient(circle at 80% 80%, #f0f0f0, transparent 45%),
26     radial-gradient(circle at 50% 50%, #f0f0f0, transparent 45%);
27   opacity: 0.45;
28   z-index: 0;
29   filter: blur(60px);
30 }
31
32 .app-shell {
33   position: relative;
34   z-index: 1;
35   width: min(960px, 95vw);
36   margin: 0 auto;
37   padding: 5px 1.5rem 0 0;
38 }
39
40
```

```
File Edit Selection View Go Run Terminal Help
# styles.css
fontend > # styles > % app-shell
40 .card {
41   background: #f0f0f0;
42   border-radius: 1.5rem;
43   padding: 2.5rem;
44   box-shadow: 0 25px 70px #f0f0f0;
45   backdrop-filter: blur(20px);
46 }
47
48 .badge {
49   display: inline-flex;
50   align-items: center;
51   gap: 0.4rem;
52   font-size: 0.85rem;
53   text-transform: uppercase;
54   letter-spacing: 0.08em;
55   color: #f0f0f0;
56   padding: 0.35rem 0.75rem;
57   border-radius: 999px;
58   background: #f0f0f0;
59 }
60
61 .badge-soft {
62   background: #f0f0f0;
63   color: #f0f0f0;
64 }
65
66 .predictor-header {
67   margin-bottom: 1.5rem;
68 }
69
70 .subtle {
71   margin: 0.35rem 0 0;
72   color: #f0f0f0;
73 }
74
75 .split {
76   display: grid;
77   grid-template-columns: minmax(0, 1.4fr) minmax(280px, 1fr);
78   gap: 2rem;
79 }
80
```

Backend:



```
1 from __future__ import annotations
2
3 import logging
4 import json
5 from pathlib import Path
6 from typing import Any, Dict
7
8 import joblib
9 import numpy as np
10 from flask import Flask, jsonify, request
11 from flask_cors import CORS
12
13 BASE_DIR = Path(__file__).resolve().parent
14 MODEL_DIR = BASE_DIR.parent / "model"
15 MODEL_PATH = MODEL_DIR / "model.joblib"
16 METRICS_PATH = MODEL_DIR / "metrics.json"
17
18 app = Flask(__name__)
19 CORS(app)
20
21 logger = logging.getLogger(__name__)
22 logging.basicConfig(level=logging.INFO)
23
24
25 def load_model():
26     if not MODEL_PATH.exists():
27         raise FileNotFoundError(
28             f"Model file not found at {MODEL_PATH}. "
29             "Run backend/train_model.py to create it."
30         )
31     payload = joblib.load(MODEL_PATH)
32     return payload["model"], payload["features"]
33
34
35 def load_metrics() -> Dict[str, float]:
36     if not METRICS_PATH.exists():
37         return {"acc": None, "r2": None}
38     with METRICS_PATH.open() as fp:
39         return json.load(fp)
```



```
1 """
2 Train and persist a simple regression model for house price prediction.
3
4 The script downloads the California Housing dataset, engineers a few features,
5 splits the data, trains a RandomForestRegressor, evaluates it, and saves the
6 model along with the feature list inside the ../model directory.
7 """
8
9 from __future__ import annotations
10
11 import json
12 from pathlib import Path
13
14 import joblib
15 import numpy as np
16 import pandas as pd
17 from sklearn.datasets import fetch_california_housing
18 from sklearn.ensemble import RandomForestRegressor
19 from sklearn.metrics import mean_absolute_error, r2_score
20 from sklearn.model_selection import train_test_split
21
22 BASE_DIR = Path(__file__).resolve().parent
23 MODEL_DIR = BASE_DIR.parent / "model"
24 MODEL_PATH = MODEL_DIR / "model.joblib"
25 METRICS_PATH = MODEL_DIR / "metrics.json"
26
27
28 def load_dataset() -> pd.DataFrame:
29     housing = fetch_california_housing(as_frame=True)
30     df = housing.frame
31     df["avg_rooms"] = df["AveRooms"]
32     df["avg_bedrooms"] = df["AveBedrms"]
33     df["house_age"] = df["HouseAge"]
34     df["median_income"] = df["MedInc"]
35     df["population"] = df["Population"]
36     df["latitude"] = df["Latitude"]
37     df["longitude"] = df["Longitude"]
38     df["target"] = df["MedHouseVal"]
39     columns = [
```

Screenshots:

The screenshot shows a web browser window with the title "House Price Predictor" and the address bar displaying "localhost:8000". The page features a dark blue background with a central white prediction form. The form is titled "House Price Predictor" and includes a sub-header "PREDICTION FORM". Below the title, a note states: "All numbers are required. The more accurate your inputs, the better the estimate." The form contains several input fields for user data: "Median Income (10k USD)" with a value of 6, "Average Rooms" with a value of 3, "Average Bedrooms" with a value of 2, "Population" with a value of 200, "House Age (years)" with a value of 2, "Latitude" with a value of 12.34, and "Longitude" with a value of 56.77. A blue button labeled "Predict price" is positioned below the input fields. To the right of the form, a dark blue box displays the "ESTIMATED PRICE" as "\$324,589" and "₹2,69,40,881", with the text "Prediction ready !" below it. At the bottom of the form, a small note reads: "Median value is returned in \$100k units (per dataset)." The browser window also shows a "Summarize" button and various icons in the top right corner.

House Price Predictor

localhost:8000

Summarize

PREDICTION FORM

House Price Predictor

All numbers are required. The more accurate your inputs, the better the estimate.

Median Income (10k USD) Average Rooms

6 3

Average Bedrooms Population

2 200

House Age (years) Latitude

2 12.34

Longitude

56.77

Predict price

ESTIMATED PRICE

\$324,589

₹2,69,40,881

Prediction ready !

Median value is returned in \$100k units (per dataset).